

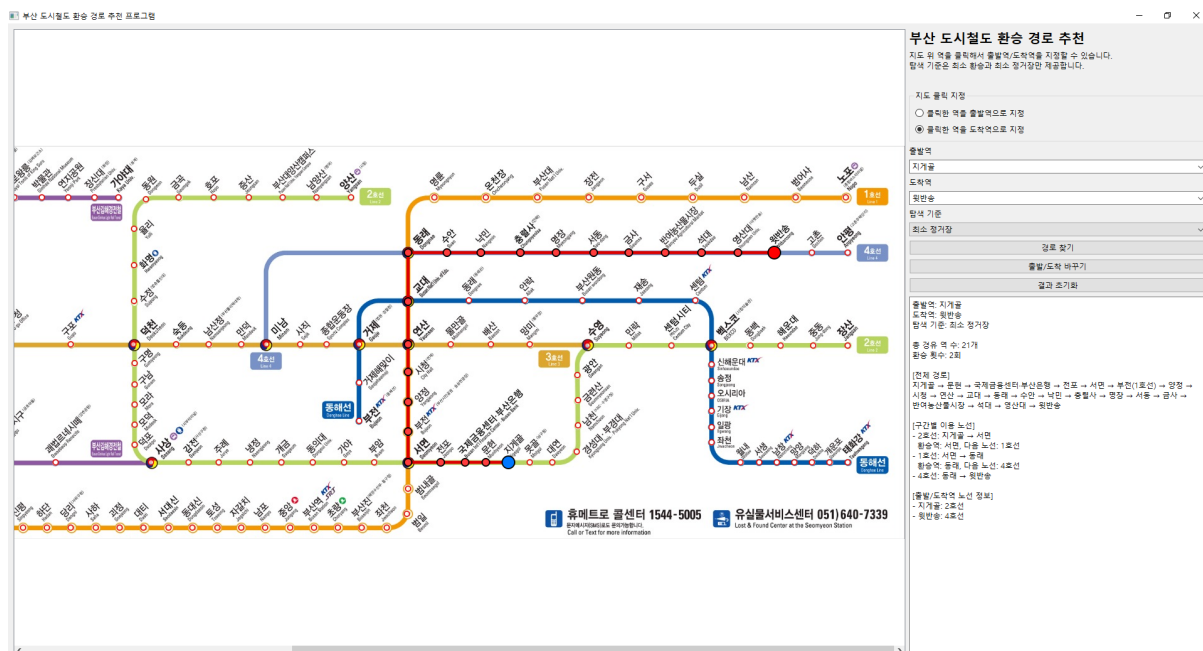
2026-1 데이터구조 최종 프로젝트

학번:202219251

이름:이시우

Github 주소:<https://github.com/siwoo1313/data-structure-lab-2026-4>

1) 최종 프로젝트 실행결과 이미지 캡처



2) 최종 프로젝트 프로그램 설명

- 프로그램 동작 설명

이 프로그램은 부산 도시철도 노선도를 이용하여 사용자가 원하는 출발역에서 도착역까지의 경로를 추천하는 GUI 프로그램이다. 프로그램은 C++와 Qt Widgets 라이브러리를 사용하여 구현하였고 모든 역과 노선 정보는 코드 내부에 저장되어 있다.

먼저 프로그램이 실행되면 부산 도시철도 노선도 이미지가 화면에 표시된다. 동시에 코드에 저장된 역 정보와 노선 정보를 바탕으로 그래프가 생성된다. 각 역은 그래프의 정점으로 저장되고, 같은 호선으로 연결된 역과 환승하여 바로 이동이 가능한 역 사이에는 간선이 연결된다. 이 때 간선에는 해당 구간이 어떤 노선에 속하는지도 함께 저장된다.

사용자는 오른쪽 입력 영역에서 출발역과 도착역을 선택할 수 있다. 또한 지도 클릭 지정 기능을 이용해 노선도 이미지 위의 역 위치를 클릭하여 출발역과 도

착역을 지정할 수 있다. 클릭된 좌표는 이미지 좌표로 변환되고 프로그램은 해당 좌표와 가장 가까운 역을 찾아 선택값으로 지정한다.

탐색 기준은 최소 환승과 최소 정거장 중 하나를 선택할 수 있다. 최소 정거장을 선택하면 BFS 탐색을 통해 지나가는 역 수가 가장 적은 경로를 찾는다. 최소 환승을 선택하면 현재 이용 중인 노선 상태를 고려하여 환승 횟수가 가장 적은 경로를 찾고, 환승 횟수가 같을 경우 정거장 수가 더 적은 경로를 선택한다.

경로 탐색이 완료되면 지도 위에 경로가 선으로 표시되고 결과창에는 전체 이동 경로와 구간별 이용 노선이 출력된다. 결과 초기화 버튼을 누르면 지도 위에 표시된 경로와 결과창 내용이 초기화된다.

- 왜 그래프 자료구조에 적합한 데이터인지?

지하철 노선도는 여러개의 역과 역 사이의 연결관계로 표시되어 있기 때문에 부산 도시철도 노선도는 그래프 자료구조로 표현하기에 매우 적합한 데이터이다.

그래프에서 정점은 하나의 역을 의미하고, 간선은 서로 직접 연결된 두 역 사이의 이동 구간을 의미한다. 예를 들어 사상역과 감전역이 서로 연결되어 있다면, 두 역은 그래프에서 하나의 간선으로 연결된다. 또한 서면, 수영, 덕천, 연산, 교대와 같은 환승역은 여러 노선이 만나는 지점이므로 그래프에서 여러 간선이 연결된 정점으로 표현할 수 있다.

지하철 경로 탐색은 결국 한 정점에서 다른 정점까지 어떤 간선을 따라 이동할 것인지를 찾는 문제이다. 따라서 최단 경로, 최소 정거장 경로, 최소 환승 경로와 같은 문제는 그래프 탐색 알고리즘을 적용하기에 적합하다.

이번 프로젝트에서 역 정보를 정점으로 저장하고, 인접한 역들을 간선으로 연결했다. 이를 통해 실제 지하철 노선도의 연결 구조를 코드 내부에서 그래프 형태로 표현할 수 있었다. 최소 정거장 경로는 모든 간선의 비용이 동일하다 보고 BFS를 적용했다. 최소 환승 경로는 현재 노선 상태를 함께 고려하는 방식으로 탐색했다.

이번 프로젝트의 데이터는 그래프 자료구조의 핵심 요소인 정점, 간선, 연결 관계를 모두 포함하고 있고 그래프 탐색 알고리즘을 적용하여 실제 사용자가 원하는 경로를 찾을 수 있다는 점에서 그래프 자료구조에 적합하다.

3) 최종 프로젝트 코드 설명

- 객체지향 구조 어떻게 구현했는지 설명

이번 프로젝트는 여러 개의 구조체와 클래스를 사용하여 객체 지향적으로 구현했다. 프로그램의 주요 구성 요소는 역 정보, 간선 정보, 경로 결과, 그래프 관리 클래스, 지도 표시 클래스, GUI 전체를 관리하는 메인 윈도우 클래스로 나눌 수 있다.

StationDef 구조체는 역 이름과 노선도 이미지 위의 좌표를 저장하기 위해 사용된다. 이 구조체는 코드에서 노선 데이터를 입력할 때 사용되고 각 역의 이름

과 x, y 좌표를 함께 저장한다.

Edge 구조체는 그래프의 간선을 표현한다. 간선에는 연결된 다음 역 번호, 해당 구간의 노선 이름, 노선 내부 번호가 저장된다. 이를 통해 단순히 역과 역이 연결되어 있다는 정보 뿐만 아니라, 어떤 노선을 이용하여 이동하는지도 확인할 수 있다.

Station 구조체는 경로 탐색 결과를 저장한다. 경로 탐색 성공 여부, 환승 횟수, 전체 경유 역 수, 경로에 포함된 역 번호 목록, 각 구간에서 이용한 노선 목록을 포함한다.

가장 핵심적인 클래스는 MetroGraph 클래스이다. 이 클래스는 부산 도시철도 데이터를 그래프 형태로 저장하고, 경로 탐색 알고리즘을 수행한다. 내부에는 전체 역 목록, 인접 리스트, 역 이름과 역 번호를 연결하는 해시 테이블, 노선 이름과 노선 번호를 연결하는 해시 테이블이 존재한다. addLine() 함수는 하나의 노선을 역 순서대로 추가하고, 인접한 역들 사이에 간선을 생성한다. searchRoute() 함수는 사용자가 선택한 탐색 기준에 따라 최소 정거장 경로 또는 최소 환승 경로를 탐색한다.

MetroMapView 클래스는 QGraphicsView를 상속하여 지도 이미지를 화면에 표시하는 역할을 한다. 이 클래스는 노선도 이미지를 표시하고, 역 좌표 점을 그리고, 경로 탐색 결과를 지도 위에 선과 원으로 표시한다. 또한 마우스 클릭 이벤트를 처리해 사용자가 지도 위에서 역을 선택할 수 있도록 한다.

MainWindow 클래스는 전체 GUI를 구성하고 사용자 입력과 프로그램 동작을 연결하는 역할을 한다. 출발역 선택 ComboBox, 도착역 선택 ComboBox, 탐색 기준 ComboBox 버튼, 결과 출력창을 생성하고 배치한다. 또한 버튼 클릭 이벤트와 지도 클릭 이벤트를 연결하여 사용자가 프로그램을 조작할 수 있도록 한다.

이처럼 이번 프로젝트는 기능별로 구조체와 클래스를 분리하여 구현하였다. 그래프 데이터 처리, 지도 표시, GUI 제어가 각각 독립적인 역할을 가지도록 구성했기 때문에 코드의 역할이 명확하고 유지보수가 쉽다.

- 각 클래스 구조 간의 관계가 어떻게 되는지 설명

이 프로그램의 클래스와 구조체는 서로 역할을 나누어 협력하는 구조로 이루어져 있다.

MainWindow 클래스는 프로그램 전체를 관리하는 중심 클래스이다. 사용자가 버튼을 누르거나 지도 위 역을 클릭하면 MainWindow가 해당 이벤트를 처리한다. 하지만 MainWindow가 직접 경로 탐색을 수행하지는 않고, MetroGraph 클래스에 경로 탐색을 요청한다.

MetroGraph 클래스는 그래프 데이터를 관리하는 클래스이다. 부산 도시철도 역과 노선 정보를 내부에 저장하고, 출발역과 도착역이 주어지면 경로 탐색 결과를 RouteResult 형태로 반환한다. MainWindow는 사용자가 입력한 역 이름을

MetroGraph에 전달하고, 반환된 결과를 GUI에 출력한다.

MetroMapView 클래스는 지도 이미지를 표시하고 경로를 시각적으로 표현하는 역할을 한다. MainWindow는 MetroGraph로부터 받은 경로 결과를 MetroMapView에 전달하고 MetroMapView는 해당 역들과 좌표를 이용해 지도 위에 선과 원을 그린다.

Station, Edge, RouteResult 구조체는 MetroGraph와 MetroMapView, MainWindow 사이에서 필요한 데이터를 담는 역할을 한다. Station은 역 이름과 좌표를 저장하고 Edge는 역 사이의 연결 관계를 저장한다. RouteResult는 탐색 결과를 저장하여 MainWindow가 결과창과 지도 표시를 구성할 수 있도록 한다.

정리하면 Metro Graph는 데이터와 알고리즘을 담당하고, MetroMapView는 지도 표시와 경로 시각화를 담당하고, MainWindow는 GUI전체 흐름을 제어한다. 이러한 구조는 객체 지향 프로그래밍의 특징인 역할 분리와 캡슐화를 잘 보여 준다.

- GUI 앱 구조를 어떻게 구성했는지 설명

이번 프로젝트는 Qt Widgets을 이용해 GUI 앱 구조를 구성했다. 전체 화면은 크게 왼쪽 지도 영역과 오른쪽 입력과 결과 영역으로 나뉘어진다.

왼쪽 지도 영역은 MetroMapView 객체로 구성된다. MetroMapView는 QGraphicsView를 상속한 클래스이고 내부적으로 QGraphicsScene을 사용하여 노선도 이미지, 역 좌표 점, 경로 표시 선과 원을 화면에 그린다. 노선도 이미지는 QPixmap을 통해 불러오고, QGraphicsScene에 추가되어 화면에 표시된다.

오른쪽 입력 영역은 QVBoxLayout을 이용했다. 이 영역에는 프로그램 제목, 사용 안내 문구, 지도 클릭 지정 RadioButton, 출발역 Comobox, 도착역 Comobox, 탐색 기준 Comobox, 경로 찾기 버튼, 출발역/도착역 바꾸기 버튼, 결과 초기화 버튼, 결과 출력창이 배치되어 있다.

출발역과 도착역 선택에는 QComboBox를 이용했다. ComboBox는 사용자가 역을 직접 선택할 수 있을 뿐만 아니라 역 이름을 입력할 수 있도록 editable 속성을 사용했다. QCompleter를 적용해 역 이름 자동완성 기능을 지원한다.

탐색 기준은 QComboBox로 구성되어 있고 사용자는 최소 환승과 최소 정거장 중 하나를 선택할 수 있다. 경로 찾기 버튼을 누르면 현재 선택된 출발역, 도착역, 탐색 기준을 바탕으로 경로 탐색이 수행된다.

결과 출력창은 QTextEdit을 이용했다. 탐색 결과가 나오면 출발역, 도착역, 총 경유 역 수, 환승 횟수, 전체 경로, 구간별 이용 노선이 텍스트로 표시된다.

버튼 동작은 Qt의 Connect() 함수를 이용하여 연결했다. 경로 찾기 버튼은 searchRoute() 함수와 연결해 경로 탐색을 수행한다. 출발역/도착역 바꾸기 버튼은 두 ComboBox의 값을 서로 교환한다. 결과 초기화 버튼은 지도 위 경로 표시를 제거하고 결과창을 초기화한다.

지도 클릭 기능은 MetroMapView의 마우스 클릭 이벤트를 이용해 구현하였다. 사용자가 지도 위를 클릭하면 클릭 위치가 이미지 좌표로 변환되고 MetroGraph의 nearestStation() 함수를 통해 가장 가까운 역을 찾는다. 이후 현재 선택된 클릭 모드에 따라 해당 역이 출발역 또는 도착역으로 지정된다.

이번 프로젝트의 GUI는 사용자가 직관적으로 지하철 경로를 탐색할 수 있도록 지도 표시 영역과 입력/결과 영역을 분리하여 구성했다. Qt Widgets의 다양한 위젯과 이벤트 연결 방식을 활용하여 사용자의 입력, 경로 탐색, 결과 표시가 하나의 흐름으로 동작하도록 구현했다.